A Comparative Study of Windows, MacOS and Linux

Harshiya Saxena
Computer Science and Engineering
SRM Institute of Science and Technology Ghaziabad, India
hs9268@srmist.edu.in

Harshit
Computer Science and Engineering
SRM Institute of Science and Technology Ghaziabad, India
hz5783@srmist.edu.in

Shruti Goel
Computer Science and Engineering
SRM Institute of Science and Technology Ghaziabad, India
sg0616@srmist.edu.in

Divyam Agarwal
Computer Science and Engineering
SRM Institute of Science and Technology Ghaziabad, India
da0617@srmist.edu.in

Mrs. Ankita Singh
Computer Science and Engineering
SRM Institute of Science and Technology Ghaziabad, India
ankitas@srmist.edu.in

***Abstract*— Desktop operating systems are becoming increasingly relevant in software development as well as in their day-to-day operation. Users have compared the three major desktop operating systems-Windows, macOS and Linux-numerically under various operational scenarios. Kernel function, interaction of background services, cache characteristics, regression caused by system update procedures, workflow, permissions granted to users, virtualization overhead, system recovery, system thermal sensitivity and stability will help comparative analyses of the three-operating system based on data from this current study. Experimental metrics will be established to analyze and evaluate the mentioned factors fairly so that clear comparison can be made amongst the three operating systems. The Background service Interference Index and the Performance Variance Stability Index are two new experimental indices created as part of this study. The results of our findings indicated that the Linux operating system had the most consistent and well-performing kernel performance and stability, Windows has superior recovery times and workflow, and macOS has the least friction with regards to user permission.**

Keywords—Windows, macOS, Linux, operating systems, performance analysis, system stability, virtualization, human–OS interaction.

I. INTRODUCTION

Desktop operating systems are commonly known for serving as the heart of computing [1],[2], in fact, they are used by many individuals for a variety of programming tasks, including but not limited to executing Virtual Machine, viewing media files, and performing all of the above with countless different programs or applications. A result of the pressure that applications continuously create through their respective demands upon system resources, it is imperative that resources are properly managed, to deliver consistent and stable, fluid, and user-friendly operating environment [1],[3].

Despite the fact that Windows, Linux and macOS are very common OS's, they are still not very many real-life comparisons of their performance capabilities in actual use. Performance issue background service usage, the effects of operating system updates upon performance and recovery stability have an impact on end-user satisfaction and dependability.

While there are similarities in the basic feature of Linux, macOS, and Windows, such as providing core functionality, the kernel design, service management, security models and user interaction methods differ greatly from one another [3],[4]. In providing for modularity and openness, much like Linux [5],[6], macOS has a tightly controlled ecosystem with an integrated security policy [7],[8]. Windows focuses on compatibility and availability by supporting numerous background services [9],[10]. These architectural differences directly influence system responsiveness and runtime efficiency. Many of the variables associated with dynamic runtime behavior, execution variability, thermal attribute, recovery characteristics and workload characteristics [11] (developer-focused) have been largely ignored in the literature, which has mainly focused on synthetic benchmarks, vulnerability assessments or qualitative usability metrics [12],[13] when evaluating operating systems.

The present research would conduct controlled experimental studies to explore these three identified operating systems [12], using a variety of metrics for measuring kernel transition delays [5],[14], background services interferences, cache behaviors, update regressions, programmer latency, permission friction, virtualization delays [13], recovery time, sensitivity to thermal throttling, and runtime stability, through two indices that will allow comparisons between the identified Operating Systems. By using multiple measurements and usability-oriented measures, the present study will provide a practical understanding of the three identified Operating Systems focusing on architectural trade-offs.

II. BACKGROUND WORK

Research on desktop operating systems has primarily focused on topics related to security, performance tests, and the usability of the operating system. The open-source characteristics of Linux have attracted a greater amount of research [5],[10]; however, Windows has been the subject of numerous studies because of the near-universal user base and widespread potential for malware. [9],[10]. There have been relatively few studies on Mac OS, which is due to the closed or proprietary nature of the operating system [7],[8].

The philosophy that governs the development of an application has been shown by many studies to affect both user behavior and application outcomes. For example, the developer's philosophy affects how Windows users are exposed to risk due to compatibility issues, how Linux users explore system customization, and how users of MacOS do so in a secure environment focused on user experience.

Research has been done on individual factors. For example, research exists on the efficiency of virtualization [13],[15], kernel structure [1],[5], and simulation based approaches [16]. However, most of this research is conducted without a cohesive experimental methodology and is based on average benchmark results. Additionally, the dynamic impacts of runtime environments have not been adequately studied [11],[12], including interference from executing background services, cache warm-up effects, regression after an update occurrence, and human interaction costs.

Table I. presents a comparative analysis of current studies to highlight limitations of current research.

TABLE I. COMPARISON OF CURRENT RESEARCH

S. No.	*Author name [Year]*	*Methodology*	*Results*	*Limitations*
1	[17] Idris et al. (2022)	Comparative study and the analysis were done of the Windows, MacOS and Linux. It was based on file systems, security, user- experience, kernel type, their compatibility and cost-effectiveness.	It was found that Windows is the most widely used OS due to its affordability and compatibility. MacOS was better in terms of security and reliability.	MacOS is costly and can be used only with Apple Hardware. While windows have higher security threats.
2	[18] Pradhan et al. (2021)	A qualitative comparison of Windows, MacOS and Linux was done. It was focussed on UI, performance, software availability and popularity among users.	Windows was offering higher compatibility and its use was easy. While MacOS gave stable performance and better UI.	Windows needs frequent updates. While MacOS lacks extensive software and gaming support.
3	[19] Hasnain et al. (2019)	Multiple criterions were considered for comparison. Example: - convenience, security, interface, recovery, boot time and cost.	Windows gave the best results for convenience and compatibility. While MacOS gave better results for reliability and security.	MacOS offered limited scalability and hardware upgrades. On the other hand, Windows was vulnerable to malware.
4	[20] Softić et al. (2022)	Vulnerability was compared using CVE and NVD databases (2015-2021) for Windows 10 and MacOS.	Windows 10 had the highest number of vulnerabilities. On the other hand, MacOS had moderate vulnerability levels.	Vulnerability counts is affected by popularity bias. These results were dependent on disclosed vulnerabilities only.
5	[21] De La Cruz (2023)	Windows, MacOS and Linux were compared based on their security features. This approach was focussed on malware, phishing and DoS attacks.	MacOS had much better built in security mechanisms. While windows required additional security layers.	No OS is 100% secure. Their effectiveness depends on user practices and updates.
6	[22] Adekotujo et al. (2020)	Comparative analysis of modern Operating Systems, for example- Windows, MacOS, Linux, Android and iOS. The comparison was based on their strengths and weaknesses.	Windows and MacOS came across as secure and reliable. Windows had an upper edge of being more user-friendly and popular.	MacOS is expensive. Windows is susceptible to malware because of larger user base,
7	[23] Kurhade et al. (2024)	Performance benchmarking of Windows was done using Geek bench (CPU and memory tests).	Windows showed better results in multi- core and memory performance on different hardware.	Results vary because of hardware configurations.
8	[24] Muhammad & Rafi (2019)	Comparative study and analysing user preference trends to help in making decisions.	Windows is preferred for general users. MacOS is better for professional and creative works.	User preferences might change over time with OS updates and additional features.

Although there are many benchmarking tools and software for individual operating system, but studies that uses same metrics across all of three is rare. Most of the researches relies on synthetic benchmarks and average scores. They tend to often overlook runtime variability, background interferences, recovery systems and how human interact with the system. This study employs a controlled experimental design setup with user-focused metrics and standard system-level.

III. METHODOLOGY

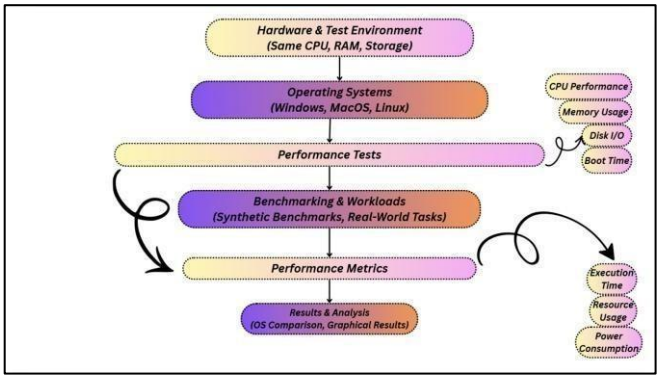

Fig.1. Methodology Flowchart

A. System Installation and Experimental Preparation

All the three operating system was deployed on the identical hardware configurations using the stable releases of their versions fetched from their official sources. Windows got installed using their installation recovery key, Linux through the recoverable DISC ISO setup file using a bootable pen-drive and macOS through the apple's recovery-based installer. Services native to the platform was retained in order to preserve real- world operating conditions.

Normalized set of evaluation tasks was applied across all the platforms, and repeated executions in a controlled runs to capture variance to make sure reliable measurements for analysis.

B. Experimental Setup

Every experiment was conducted locally and in a controlled conditioned system, so as to make sure consistency and reproducibility.

Self-written automation shell and python scripts were used to perform evaluation tasks, performance measurements tasks across the platforms. These scripts were setup before all the tests were started to perform and all the recording files were also setup.

Many times, executions were performed for every task, and results were getting automatically logged and stored in a CSV file for subsequent analysis.

Overall whole setup enabled standardized result collection while making almost zero manual or human interaction and biases on experimental basis.

C. Analysis Procedure

This study and the evaluations were conducted strictly under the controlled environment and conditions across the platforms, with the analysis being conducted according to the methodology described below:

- Every platform and operating was being rebooted and reset in order to clear cache and any background activity.
- Normalized measuring tasks were being executed linearly in a sequence under identical workloads across the platforms.
- Self-written scripts used to record the execution time, recovery behavior of each run and resource utilization.
- Built-in tools and systems were used to measure visualization and thermal parameters.
- Every result and output was being stored in a structured CSV type file to ensure consistent parsing and statistical processing.
- Dedicated aggregation scripts were applied to integrate the collected results and to compute the mean performance values and runtime variance.
- The final data after being processed thoroughly, then organized into a tabular format for further comparative and visualization assessment.

TABLE II. DATA FEATURE EXTRACTION

Feature	Windows	macOS	Linux
System Metadata	OS Version, Build, NT Kernel	OS Version, Darwin Kernel	Distro Version, Kernel
Kernel Performance	Syscall Latency, Context Switch	Syscall Latency, Context Switch	Syscall Latency, Context Switch
Background Activity	Services, Startup Tasks	Launch Daemons, Agents	systemd Services
Developer Workflow	Compile, Docker, Git	Compile, Docker, Git	Compile, Docker, Git
Security Interaction	UAC Prompts	Permission Dialogs	sudo Requests
Runtime Stability	Variance Across Runs	Variance Across Runs	Variance Across Runs

D. Evaluation Metrics

To make sure fair comparison across the platforms, all the raw measurements were being standardized before the analysis. Platform-specific system indicators was also mapped to common representations, for allowing consistent evaluation across the same environments. For e.g., all the background activities was normalized by using active service counts, while user security interactions was being captured using permission prompts and confirmation delays. Every test was done many times, and average of all the values was used to reduce transient fluctuations and to capture runtime variability.

The Background service interference index (BSII) is defined by us in order to specify in numerical terms how background processes affects the foreground performance:

$$BSII = \frac{\text{Foreground Task Slowdown (\%)}}{\text{Number of Active Background Services}}$$

Cold Cache Degradation (CCD) is calculated by comparing execution time immediately after rebooting with repeated runs:

$$CCD = \frac{Cold - Warm}{Warm}$$

Virtualization Overhead Differential (VOD) is used to measure loss in performance whenever workloads being executed in virtualized environments:

$$VOD = \frac{Native - Virtualized}{Native}$$

Runtime stability was measured using variance across all the repeated executions and runs, lower variance suggests greater consistency:

$$Stability = Var(R_1, R_2, \ldots, R_n)$$

Thermal Throttling Sensitivity (TTS) is measured as the relative performance drop under elevated temperature conditions:

$$TTS = \frac{P_{baseline} - P_{thermal}}{P_{baseline}}$$

Finally, a composite Performance Variance Stability Index (PVSI) was derived by normalizing stability values to a 0–100 scale:

$$PVSI = 100 \times \frac{S - min(S)}{max(S) - min(S)}$$

Together, these standardized metrics capture low-level performance characteristics, system resilience, virtualization efficiency, and user interaction costs, providing a comprehensive basis for comparing operating system behavior under realistic conditions.

Proposed Evaluation Metrics

S.No	*Evaluation Metric*	*What It Measures*
1	Kernel-to-user space transition overhead	Average latency of system calls reflecting low-level OS efficiency
2	Background Service Interference Index (BSII)	Foreground performance degradation normalized by background services
3	Cold vs Warm cache performance degradation	Effectiveness of file system & memory caching
4	Update-Induced performance regression	Performance changes before & after OS updates
5	Developer workflow latency	Suitability for development tasks (compile, git, containers)
6	Permission Friction Score (PFS)	Human-OS interaction cost due to permission prompts
7	Virtualization overhead differential	Performance penalty of virtualized vs native execution
8	System recovery time after forced failure	OS resilience & recovery capability
9	Thermal throttling sensitivity	Performance degradation under thermal stress
10	Cross-OS Performance variance stability index	Consistency & stability across repeated runs

IV. RESULTS

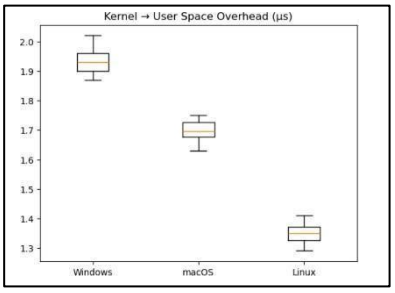

Fig.2. *Cross-Platform Kernel Transition Overhead*

Fig. 2 shows kernel-to-user space transition latency across Windows, macOS, and Linux. Windows records the highest overhead (1.87–2.02 µs, mean ≈1.93 µs), macOS exhibits moderate latency (1.63–1.75 µs, mean ≈1.70 µs), while Linux consistently achieves the lowest values (1.29–1.41 µs, mean ≈1.35 µs).
The efficiency of the Linux kernel-user transition results looks to be higher than macOS and Windows because the Linux kernel has a light weight design [5],[14]. Even though macOS has good, consistent performance, Windows has higher overhead costs due to increased activity from the service layer [9],[10]. The performance of the kernel architecture of the respective OSes is illustrated in the way each OS architecture performs at the system level.

Fig. 3 presents the Background Service Interference Index (BSII) across Windows, macOS, and Linux. Windows shows the highest interference (0.39–0.47, mean ≈0.43), followed by macOS (0.31–0.36, mean ≈0.34), while Linux records the lowest values (0.21–0.26, mean ≈0.23).

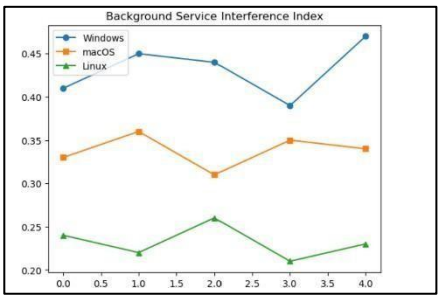

Fig.3. *Background Service Interference Index (BSII)*

The results indicate that Windows has more significant reductions in performance than the other operating systems do because they experience more runtime contentions. macOS has a moderate level of isolation relative to Linux's characteristic low amount of interference; this indicates that the design of Linux was placed mainly on performing compute-intensive jobs while placing less emphasis on providing service availability like that provided by Windows.

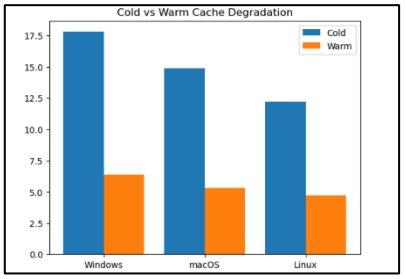

Fig.4. *Cold Cache vs Warm Cache- Performance Degradation*

Fig. 4 shows the impact of cold cache and warm cache, while we are running the tasks in different platforms. Windows showed the largest performance drop with a cold cache (17.8%), followed by MacOS(14.9%) and Linux (12.2%). When the cache was warm, we observed a performance upgradation for all the systems. However, windows still showed a higher degradation(6.4%), followed by MacOS(5.3%) and Linux (4.7%).

These results means that Linux stabilizes performance more quickly because it reuses the cached data more efficiently [1],[2]. MacOS showed moderate improvement because of background processes and filesystem overhead. This shows clear differences in memory management efficiency across the different Oss.

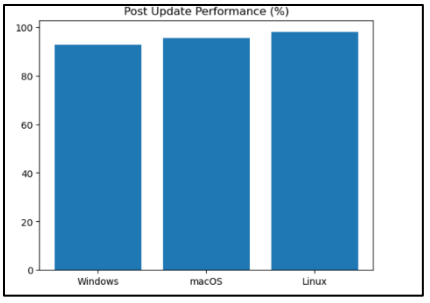

Fig.5. *Update-Induced Performance Regression*

Fig. 5 shows that how the system performance changed after updating the operating system. Linux gave the best results after the update, maintaining 97.9% of its original performance, followed by macOS with 95.4% and Windows had the biggest drop, falling to 92.7%.

These results highlighted that the updates in Linux have minimal disruption effects on runtime, likely because of its modular update mechanisms. MacOS showed moderate regression, indicating additional security validation processes, whereas Windows suffered

the highest performance impact due to its extensive background reconfiguration and service restarts. This highlighted that how the update strategies can directly impact post update system performance.

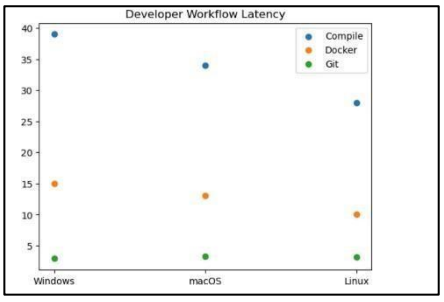

Fig.6. *Developer Workflow Latency*

Fig.6. indicates how long common developer tasks take on different platforms. It includes code compilation, container setup and Git operations. Linux was the fastest for compiling the codes (28 s) and for starting the docker containers (10 s). MacOS performed next best and Windows took more time for these tasks. Windows performed slightly better in Git related tasks (2.9 s).

Overall, this result indicated that Linux is better for heavy development tasks that need more computing power. Windows will be a better choice for simpler and file-based operations. While macOS will offer balanced and average performance. It shows that the productivity of the developer depends upon the type of workload and the operating system being used.

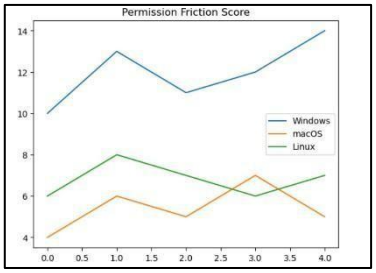

Fig.7. P*ermission Friction Score*

Fig. 7 shows the amount of time the users need to spend in dealing with the permission requests on different platforms. MacOS has the lowest time which makes it the smoothest to use. Linux has moderate delay while Windows takes the longest time.

It means that MacOS gives easier and faster security interactions, Linux needs some user input but gives moderate results. While Windows causes more delays because of frequent permission requests which negatively affects the user experience.

F. Virtualization Overhead Differential

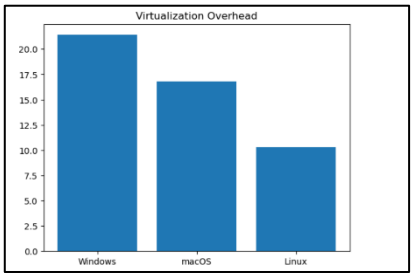

Fig.8. *Virtualization Overhead Differential*

Fig. 8 is comparing the impact of virtualization on the performance across the different platforms. Linux had the lowest loss in its performance (10.3%), which means that it can handle virtualization most efficiently. MacOS had moderate loss (16.8%). While windows had the highest impact on the performance (21.4%).

These results indicate that Linux supports virtualization very efficiently because it has strong kernel level integration [13],[15]. While MacOS and Windows have extra abstraction layers which add the overhead and thus reduces the performance of virtualized workloads.

G. System Recovery Time After Forced Failure

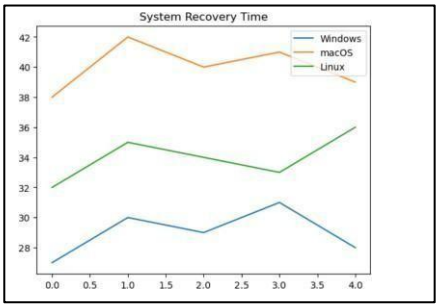

Fig.9. *Differences in the System Recovery times*

Fig. 9 shows that how long it takes for each system to recover after a controlled failure. Windows was able to recover the fastest, taking 27-31 seconds. Linux was able to recover in 32-36 seconds while MacOS took the longest to recover (38-42 s).

This suggests that Windows restarts the services more aggressively [9],[10]. While MacOS took more time because of its extra security checks and system validation during the recovery.

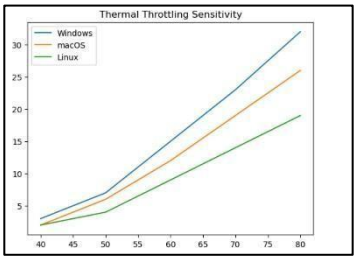

Fig.10. *Thermal Throttling Sensitivity*

Fig. 10 shows how system performance changes as the temperature increases. Windows was affected the most. Its performance dropped by 32% at 80 °C. MacOS showed a smaller drop of 26%. While Linux was the least affected with only 19% reduction.

This means that Linux is able to handle the heat better and keeps the performance more stable [12]. MacOS showed moderate performance throttling while the performance of windows degraded the most as temperature was increased.

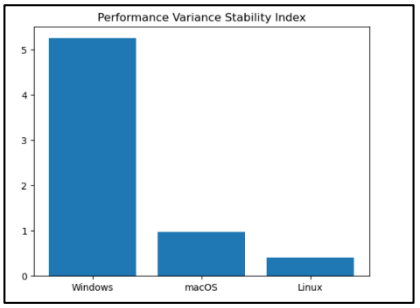

Fig.11. *Performance Variance Stability Index*

Fig. 11 compares how consistent the system performance is when the same task is run multiple times. Windows showed the most variation. It means its performance changes a lot between different runs. MacOS showed moderate variation. While Linux was the most consistent, having small changes between different executions.

This indicates that Linux provides more stable results for repeated workloads. While windows show a great fluctuation likely because of the background processes and task scheduling differences.

V. CONCLUSION AND FUTURE WORK

Our current study is based on all the three operating systems and we have used both system metrics and those metrics also that affect the user experience. The results have indicated that Linux performs uniformly with less system overhead [5],[10], very less background interference and with more stable runtime behavior. Windows is able to recover faster in case of the failures and macOS is good for smoother interactions where permissions are needed.

We also developed new evaluation parameters like BSII, PVSI. They helped us to find out how stable and easy it is to use something. They show that how an operating system works when we are using it on a daily basis is just as important as how fast it is, especially for people who use it a lot on a daily basis such as developers, testers and so on.

In future, we plan to test the system on different types of computers on the basis of their hardware specifications and see how they work when being used for a long time. We also aim to conduct a survey to understand that how the different users actually think while working with them. We will also try it on the computers which are based on ARM based chips, Snapdragon chipset, Apple's iPhone chipsets. We want to study and go deeper into system call behavior and plan to take a closer look at the kernel.

References

[1] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Pearson Education, 2015

[2] J. Corbet, A. Rubini, and G. Kroah-Hartman, Linux Device Drivers, 3rd ed. O'Reilly Media, 2005

[3] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Pearson, 2018

[4] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Wiley, 2018

[5] R. Love, Linux Kernel Development, 3rd ed. Addison-Wesley, 2010

[6] D. P. Bovet and M. Cesati, Understanding the Linux Kernle, 3rd ed. O'Reilly Media, 2005

[7] Apple Inc., "macOS Security Overview", Apple Platform Security Documentation, 2023

[8] A. Singh, macOS X Internals: A Systems Approach. Addison-Wesley, 2006

[9] Microsoft Corporation, Windows Internals, 7th ed. Microsoft Press, 2017

[10] M. Russinovich, D. Solomon, and A. Ionescu, Windows Internals, Part 1, 7th ed. Microsoft Press, 2017

[11] A. S. Tanenbaum and A. S. Woodhull, Operating Systems: Design and Implementation, 3rd ed. Pearnson, 2006

[12] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2019

[13] A. Barham et al., "Xen and the art of virtualization", in Proc. 19th ACM Symp. Operating Systems Principles (SOSP), 2003, pp. 164-177

[14] M. Desnoyers et al., "The LTTng tracer: A low impact performance monitoring tool for Linux", in Proc. Ottawa Linux Symposium (OLS), 2006

[15] P. Barham and B. Dragovic, "Performance isolation and resource Management in operating systems", ACM SIGOPS Operating Systems Review, vol. 37, no. 5, pp. 261-273, 2003

[16] S. McCanne and S. Floyd, "The ns network simulator", Software-Practice and experience, vol. 27, no. 3, pp. 299-327, 1997

[17] S. Idris, M. Khan, and A. Rahman, "Comparative study of Windows, macOS and Linux operating systems based on security, Kernel architecture and user experience" International Journal of Computer Applications, vol. 184, no. 12, pp. 15-20, 2022

[18] P. Pradhan, R. Sharma and S. Mishra, "A qualitative comparison of Windows, macOS and Linux operating systems based on user interface, performance and software availability", International Journal of Advanced Computer Science and Applications, vol. 12, no. 7, pp. 421-427, 2021

[19] M. Hasnain, A. Ali and S. Ahmed, "Comparative analysis of Modern desktop operating systems based on usability, security and Performance", International Journal of Computer Science and Information Security, vol. 17, no. 5, pp. 210-216, 2019

[20] A. Softić, E. Begović and A. Kovaćević, "Vulnerability analysis of Windows and macOS using CVE and NVD databases", International Journal of Cyber Security and Digital Forensics, vol. 11, no. 2, pp. 95-102, 2022

[21] J. De La Cruz, "Security comparison of Windows, macOS and Linux operating systems against malware, phishing and denial-of-service attacks", Journal of Information Security Research, vol. 14, no. 1, pp. 33-41, 2023

[22] A. Adekotujo, O. Adeyemi and K. Olatunji, "Comparative Analysis of modern operating systems: Windows, macOS, Linux, Android and iOS", International Journal of Computer Science and Network Security, vol. 20, no. 9, pp. 85-92, 2020

[23] P. Kurhade, S. Patil and R. Deshmukh, "Performance benchmarking of Windows operating system using Geekbench CPU and memory evaluation", International Journal of Computer Engineering and Technology, vol. 15, no. 1, pp. 44-50, 2024

[24] A. Muhammad and M. Rafi, "Analysis of user preference Trends in operating system selection for general and professional use", International Journal of Computer Applications, vol. 178, no. 22, pp. 12-18, 2019